

Python Cheat Sheet

FROM BASIC SYNTAX TO APP TESTING

Built for quick revision, coding practice, and test-day lookup.

SECTION	YOU GET
1. Python Core	Syntax, variables, types, operators, input/output
2. Logic & Data	if/elif/else, loops, lists, tuples, sets, dicts, strings
3. Building Blocks	Functions, modules, files, exceptions, OOP
4. Testing	assertions, unittest, pytest, API/app tests, mocking
5. Speed Review	Patterns, traps, debugging checklist, mini practice

Mental model: Python code is data + logic + reusable behavior. Testing checks that the behavior matches what you intended, including failure cases.

Usage: Read the syntax, understand the tiny example, then close the sheet and reproduce it from memory. That is where the brain actually does the gym reps.

```
# first program
name = input("Name: ")
print(f'Hello, {name}!')
```

Core rule: indentation is syntax. Spaces are not decoration.

Python Core Syntax

Concept	Syntax / Example
Comment	# this is a comment
Variable	x = 10
Print	print("Hello")
Block	if x > 5: print("big")
Multiple values	a, b = 10, 20
Swap	a, b = b, a

```
x = 10
y = 3
print(x + y)
print(type(x))
```

Naming

- Use snake_case: total_marks, user_name, final_amount.
- Constants are conventionally UPPER_CASE: MAX_RETRIES = 3.
- Do not use Python keywords as variable names: if, for, class, def, return, etc.

Truth values

Expression	Result
bool(0), bool("")	False
bool(1), bool("hi")	True
None	absence of a value; treated as False in conditions
[] / {} / set()	False when empty

Trap: Python uses True/False with capital T/F. false, true and null are not Python booleans.

Test instinct: Before testing a function, identify normal input, boundary input, bad input, and missing input.

Variables, Types & Operators

Type	Example	Typical use
int	25	counts, age, indexes
float	19.99	measurements, prices
str	"hello"	text
bool	True	conditions / flags
list	[1, 2, 3]	ordered, mutable collection
tuple	(1, 2)	ordered, immutable collection
set	{1, 2, 3}	unique values
dict	{"id": 1}	key-value data

Operators

Category	Operators
Math	+ - * / // % **
Comparison	== != > < >= <=
Logic	and or not
Membership	in not in
Identity	is is not
Assignment	= += -= *= /= %=

```
10 / 3 # 3.333...
```

```
10 // 3 # 3
```

```
10 % 3 # 1
```

```
2 ** 3 # 8
```

Critical: Use == to compare values. Use = to assign. Use is mainly for identity checks such as x is None.

Testing example: assert 10 % 5 == 0

Input, Output & Type Conversion

```
name = input("Name: ")
age = int(input("Age: "))
price = float(input("Price: "))
print(f'{name} is {age} years old')
```

Convert safely

Need	Use
Text -> integer	<code>int("42")</code>
Text -> float	<code>float("3.14")</code>
Anything -> text	<code>str(42)</code>
Anything -> boolean	<code>bool(value)</code>
Text -> list	<code>list("abc")</code> # ['a', 'b', 'c']

Formatting

```
name = "Bruce"
score = 92
print(f'{name}: {score}%')
print(f'{3.14159:.2f}')
```

Common input bugs

- `input()` always returns a string. "5" + "2" becomes "52".
- Blank input may need validation before converting.
- `float("abc")` raises `ValueError`.

```
raw = input("Enter age: ").strip()
if raw.isdigit():
    age = int(raw)
else:
    print("Invalid age")
```

Testing angle: Try empty text, spaces, letters where a number is expected, negative numbers, and very large numbers.

Conditions: if / elif / else

```
age = 20

if age >= 18:
    print("adult")
elif age >= 13:
    print("teen")
else:
    print("child")
```

Compound conditions

```
if age >= 18 and is_citizen:
    print("eligible")

if username == "admin" or is_owner:
    print("access")

if not is_blocked:
    print("continue")
```

Nested logic

```
if age >= 18:
    if is_citizen:
        print("eligible")
    else:
        print("citizenship required")
else:
    print("underage")
```

Boundary checklist

Rule	Test values
age >= 18	17, 18, 19
score > 90	89, 90, 91
price <= 1000	999, 1000, 1001

Classic bug: For an exact condition, do not accidentally write > when the rule says >=. Boundary values are where bugs hide like little unemployed goblins.

Loops: for / while / break / continue

```
for i in range(5):
    print(i)
```

```
# 0 1 2 3 4
```

Pattern	Meaning
range(n)	0 to n-1
range(start, stop)	start to stop-1
range(start, stop, step)	move by step
break	exit loop immediately
continue	skip current iteration

```
for i in range(1, 6):
    if i == 3:
        break
    print(i)
```

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

while loop

```
count = 0
while count < 3:
    print(count)
    count += 1
```

Search pattern

```
nums = [4, 7, 9, 12]
target = 9

for x in nums:
    if x == target:
        print("found")
        break
else:
    print("not found")
```

Trap: A while loop needs a state change. Forgetting `count += 1` can create an infinite loop.

Lists, Tuples, Sets & Dictionaries

```
nums = [10, 20, 30]
nums.append(40)
nums[0] = 99
print(nums[1:3])
```

Structure	Key operations
list	append(), insert(), remove(), pop(), sort(), len()
tuple	indexing, slicing, count(), immutable
set	add(), remove(), discard(), union, intersection
dict	get(), keys(), values(), items(), update()

```
user = {"name": "Bruce", "age": 28}
print(user["name"])
print(user.get("city", "Unknown"))
```

Loop through collections

```
for item in nums:
    print(item)

for key, value in user.items():
    print(key, value)
```

Comprehensions

```
squares = [x*x for x in range(5)]
evens = [x for x in nums if x % 2 == 0]
```

Testing angle: For collections test empty, one item, duplicates, invalid index/key, and expected ordering.

Strings: The Daily Driver

```
text = " Hello Python "
print(text.strip())
print(text.lower())
print(text.upper())
print(text.replace("Python", "World"))
```

Task	Syntax
Length	<code>len(s)</code>
Contains	<code>"py" in s</code>
Split	<code>s.split(",")</code>
Join	<code>', '.join(items)</code>
Starts/ends	<code>s.startswith("Hi") / s.endswith("!")</code>
Slice	<code>s[1:5], s[::-1]</code>

Parsing user input

```
raw = "apple,banana,apple"
items = [x.strip() for x in raw.split(",")]
unique = set(items)
```

Useful checks

```
s.isdigit()
s.isalpha()
s.isspace()
s.lower() == "yes"
```

Edge cases: Empty string, extra spaces, different casing, punctuation, Unicode characters, and unexpected separators.

Golden rule: Normalize input early, then validate. It keeps the rest of the program simpler.

Functions: Reuse the Logic

```
def add(a, b):
    return a + b

result = add(2, 3)
```

Feature	Example
Default argument	def greet(name="Guest"):
Keyword argument	greet(name="Bruce")
Multiple returns	return total, average
Variable args	def f(*args):
Keyword args	def f(**kwargs):
Docstring	def add(a,b): """Return the sum."""

Scope

```
x = 10

def demo():
    x = 5
    return x

print(x)    # 10
print(demo()) # 5
```

Testing functions

```
def is_even(n):
    return n % 2 == 0

assert is_even(4) is True
assert is_even(5) is False
```

Mental model: A good function has a clear input -> transformation -> output. The smaller that contract, the easier it is to test.

Trap: return gives a value back; print only displays a value. Tests usually need return values, not screen output.

Exceptions & Error Handling

```
try:
    age = int(input("Age: "))
except ValueError:
    print("Enter a number")
else:
    print("Valid input")
finally:
    print("Done")
```

Keyword	Role
try	code that may fail
except	handle a specific error
else	runs when no exception occurs
finally	runs either way
raise	create an exception intentionally

```
def withdraw(balance, amount):
    if amount < 0:
        raise ValueError("amount must be >= 0")
    if amount > balance:
        raise ValueError("insufficient funds")
    return balance - amount
```

Common exceptions

Exception	Typical cause
ValueError	wrong value type/format
TypeError	wrong operation/type
IndexError	list index out of range
KeyError	missing dictionary key
ZeroDivisionError	division by zero
FileNotFoundError	missing file

Testing angle: Do not only test that good input works. Test that bad input fails in the expected way.

Files, Modules & Packages

```
with open("data.txt", "r", encoding="utf-8") as f:
    text = f.read()
```

```
with open("out.txt", "w", encoding="utf-8") as f:
    f.write("hello")
```

Mode	Meaning
r	read
w	write; replaces existing file
a	append
rb / wb	binary read / write

Imports

```
import math
print(math.sqrt(16))

from pathlib import Path
p = Path("data.txt")
print(p.exists())
```

Why with?

A context manager closes the file automatically, even when an error occurs. That makes resource handling safer and tests less flaky.

App/testing note

```
# requirements example
pytest
requests

# import in code
import requests
```

Trap: Relative paths depend on the working directory. For reliable apps/tests, build paths explicitly with `pathlib`.

OOP Basics: Class, Object, Method

```
class User:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def is_adult(self):
        return self.age >= 18

user = User("Bruce", 28)
print(user.is_adult())
```

Concept	Meaning
class	blueprint/type
object	instance of a class
__init__	constructor-like initializer
self	current object instance
attribute	data stored on object
method	function attached to a class

Testing an object

```
def test_user_age():
    user = User("A", 17)
    assert user.is_adult() is False

# also test boundary
def test_user_boundary():
    assert User("B", 18).is_adult() is True
```

Keep it sane: You do not need OOP everywhere. Use a class when data and behavior naturally belong together.

Testing Mindset & Assertions

What are you testing?

Layer	Question
Unit	Does one function/class behave correctly?
Integration	Do components work together?
API	Does the service return the expected response?
UI / App	Can the user flow complete successfully?
Regression	Did a new change break old behavior?

AAA pattern

```
# Arrange -> Act -> Assert
user = User("A", 20)    # Arrange
result = user.is_adult() # Act
assert result is True    # Assert
```

Assertions

```
assert x == 10
assert x != 0
assert x > 0
assert "py" in text
assert value is None
```

Good test: Has one clear reason to fail and checks behavior, not implementation details.

Bad test: Only checks that code runs. A test that cannot fail is basically decorative parsley.

Boundary checklist

- Minimum valid
- Maximum valid
- Just below minimum
- Just above maximum
- Empty/null
- Wrong type/format
- Duplicate/repeated data

unittest: Built-in Python Testing

```
import unittest

class TestMath(unittest.TestCase):
    def test_add(self):
        self.assertEqual(2 + 3, 5)

    def test_even(self):
        self.assertTrue(4 % 2 == 0)

if __name__ == "__main__":
    unittest.main()
```

Assertion	Use
<code>assertEqual(a,b)</code>	values should match
<code>assertNotEqual(a,b)</code>	values should differ
<code>assertTrue(x)</code>	x is truthy
<code>assertFalse(x)</code>	x is falsey
<code>assertIsNone(x)</code>	x is None
<code>assertRaises(E, fn)</code>	fn should raise E

```
with self.assertRaises(ValueError):
    int("abc")
```

Run

```
python -m unittest
python -m unittest discover
```

When useful: unittest is built in and requires no extra dependency. It is a solid baseline for coursework and standard-library projects.

pytest + App/API Testing

```
# test_app.py
import pytest

def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5

def test_division_error():
    with pytest.raises(ZeroDivisionError):
        1 / 0
```

Useful pytest commands

```
pytest
pytest -q
pytest test_app.py
pytest -k "login"
pytest -x
```

API request pattern

```
import requests

response = requests.get("https://example.com/api/users", timeout=5)
assert response.status_code == 200
data = response.json()
assert isinstance(data, list)
```

What to test in an API

Check	Example
Status	200 / 201 / 400 / 401 / 404 / 500
Schema	required keys exist; correct types
Body	expected fields/values
Headers	content type, auth behavior
Latency	response completes within expected limit
Errors	invalid input returns useful error response

Important: Network tests can be flaky or slow. For unit tests, mock the network instead of calling a real server every run.

Mocking, Debugging & Last-Minute Review

Mock external dependencies

```
from unittest.mock import Mock

api = Mock()
api.get_user.return_value = {"id": 1, "name": "A"}

user = api.get_user(1)
assert user["id"] == 1
api.get_user.assert_called_once_with(1)
```

Debug fast

- Read the traceback from bottom upward; the last exception is usually the important part.
- Print/check the values immediately before the failing line.
- Reduce the problem to the smallest input that still fails.
- For tests, reproduce the failure with one focused test before running the full suite.

Common Python traps

Trap	Remember
input()	returns str
= vs ==	= assigns; == compares
/ vs //	/ true division; // floor division
list aliasing	b = a does not copy the list
mutable default	avoid def f(x=[]):
dict key missing	use get() when absence is valid
is vs ==	identity vs value equality
range(n)	stops before n

5-second test checklist

- Happy path
- Boundary values
- Invalid input
- Empty/null input
- Expected exception
- Side effect / API response
- Regression case

Mini practice

- 1) Write is_positive(n).
- 2) Write is_divisible_by_5(n).
- 3) Find a target in a list using a loop + break.
- 4) Write discount(bill) with boundaries.
- 5) Test each function with 2 normal + 2 edge cases.

Final mental model: Write the smallest correct behavior first. Then attack it with inputs that are normal, extreme, invalid, and surprising. That is testing.